

Documentación de Arquitecturas

Propósitos · Modelo 4+1 (Kruchten) · C4 (Brown) · enfoque híbrido · ADR

Ingeniería de Software II · ITBA · 2026-1C · Repaso conceptual

RESUMEN

Documentar arquitectura es conservar el **porqué** y comunicar el **qué** a cada stakeholder. Dos esquemas canónicos: **4+1** (Kruchten) organiza por **tipo de stakeholder** en vistas ortogonales; **C4** (Brown) organiza por **nivel de zoom**. Las decisiones significativas se registran aparte en **ADRs**. La cátedra recomienda enfoque **híbrido** y nivel mínimo necesario.

§1 Por qué documentar

Una sola vista no alcanza para describir arquitectura: cada stakeholder necesita ver dimensiones distintas del mismo sistema (source: modelo-4-mas-1.md). Documentar sirve a varios propósitos:

- **Entender** — un nuevo desarrollador reconstruye cómo llegó el sistema a su forma actual.
- **Validar** — chequear que la arquitectura satisface los atributos de calidad y requisitos.
- **Criticar** — exponer la arquitectura a revisión (ej. evaluaciones tipo ATAM).
- **Proponer** — comunicar una arquitectura candidata y sus trade-offs.
- **Implementar** — guiar a quienes construyen el sistema.

FUENTE

*Las arquitecturas se entienden a través de las **decisiones que las moldearon**, no del código que las materializa. Sin registro, el "porqué" se evapora con la rotación del equipo (source: adr-architecture-decision-record.md).*

REGLA DE ORO

Nivel mínimo necesario: producí solo los artefactos que alguien va a leer. Si nadie va a usar una vista o un diagrama, no lo produzcas (source: modelo-4-mas-1.md; c4-model.md).

§2 Modelo 4+1 (Kruchten)

DEFINICIÓN

4+1: modelo de Kruchten (1995, IEEE Software, "Architectural Blueprints — The 4+1 View Model") que organiza la documentación en cuatro vistas **ortogonales** más una transversal (Escenarios) que las ata (source: modelo-4-mas-1.md).

Eje organizador: tipo de stakeholder. Cada vista existe para una audiencia y responde una pregunta distinta.

TABLA 1. Las cuatro vistas + escenarios

Vista	Qué muestra	Audiencia	Notación típica	Pregunta
Lógica	Estructura funcional: clases, paquetes, módulos, responsabilidades	Analistas, diseñadores, devs	UML Class / Package	¿Qué hace?
Proceso	Runtime: procesos, hilos, concurrencia, sincronización	Integradores, perf, sysadmins	Activity, Sequence, BPMN	¿Cómo se comporta al correr?
Desarrollo	Organización del código: módulos, libs, repos, empaquetado	PMs, devs, release managers	Component, Package, árbol de dirs	¿Cómo se organiza el código?
Física	Mapping software→hardware: nodos, redes, deployment	Infra, SREs, arq. de red	Deployment, topología cloud	¿Dónde corre?
+1 Escenarios	Use cases representativos que cruzan las 4 vistas	Todos (vista que "ata")	Use Case, secuencias específicas	¿Es coherente todo?

Atributos de calidad por vista: Proceso → performance, throughput, escalabilidad, tolerancia a fallos; Desarrollo → maintainability, reusability, portability; Física → availability, scalability, fault tolerance (source: modelo-4-mas-1.md).

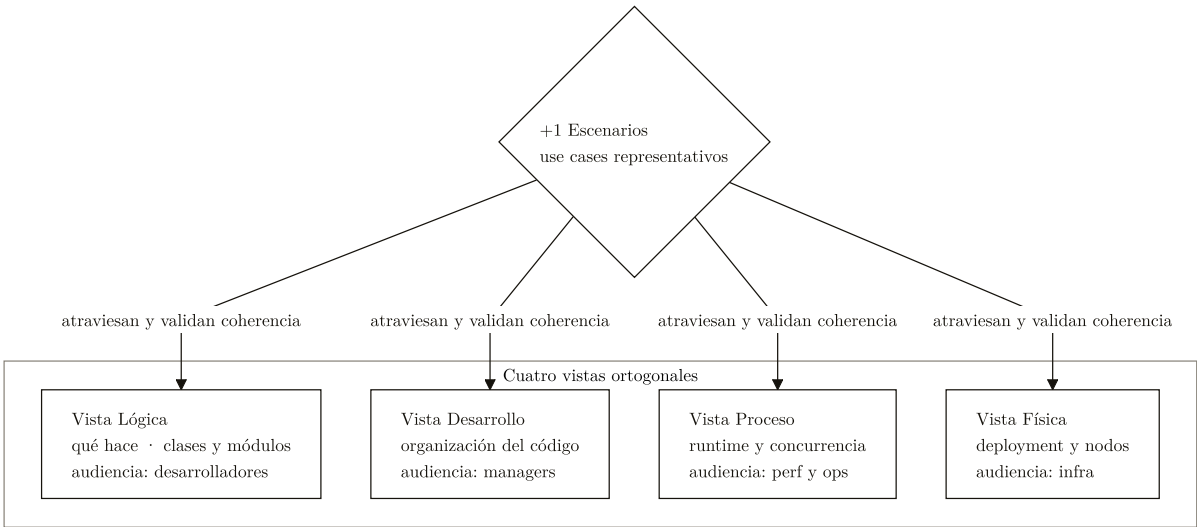


FIGURA 1. Modelo 4+1 de Kruchten: cuatro vistas ortogonales, cada una para una audiencia, atadas por los escenarios que las cruzan y validan su consistencia.

REGLA DE ORO

Reglas de oro 4+1: (1) cada vista para una audiencia real; (2) las vistas deben ser **consistentes** entre sí — un componente de Desarrollo aparece en Física y tiene comportamiento en Proceso; (3) los **escenarios validan**: si no podés trazar un escenario end-to-end, las vistas están desalineadas; (4) solo lo arquitectónicamente **significativo**, no cada clase (source: modelo-4-mas-1.md).

CUÁNDO NO • CUIDADO

Riesgos: es **UML-céntrico** y el tooling UML está rezagado para microservicios/cloud/event-driven; puede degenerar en documentación ritual (producir las 4 vistas "porque corresponde" aunque nadie las lea); no traza explícitamente atributos de calidad — hay que añadirlo a mano (source: modelo-4-mas-1.md).

§3 C4 Model (Brown)

DEFINICIÓN

C4: sistema de diagramación de Simon Brown (*Software Architecture for Developers*, ~2018) que estructura la documentación en cuatro **niveles de zoom anidados**: Context, Containers, Components, Code (las cuatro C) (source: c4-model.md).

Eje organizador: nivel de zoom. El lector elige a qué nivel mirar según su pregunta.

TABLA 2. Los cuatro niveles (zoom de afuera hacia adentro)

Nivel	Qué muestra	Audiencia	Objetivo
C1 • Context	El sistema como 1 caja + personas + sistemas externos	Cualquiera (técnico o no)	¿Cómo encaja en el mundo?
C2 • Containers	Unidades deployables (web apps, APIs, DBs, cachés, colas) + tecnología + comunicación	Técnicos (devs, ops)	¿Qué piezas deployables lo componen y cómo hablan?
C3 • Components	Módulos lógicos dentro de un container (controllers, services, repos, adapters)	Devs del sistema	¿Cómo se estructura el código del container?
C4 • Code	Clases, interfaces, relaciones concretas — típicamente UML	Devs que tocan ese componente	¿Estructura interna del componente?

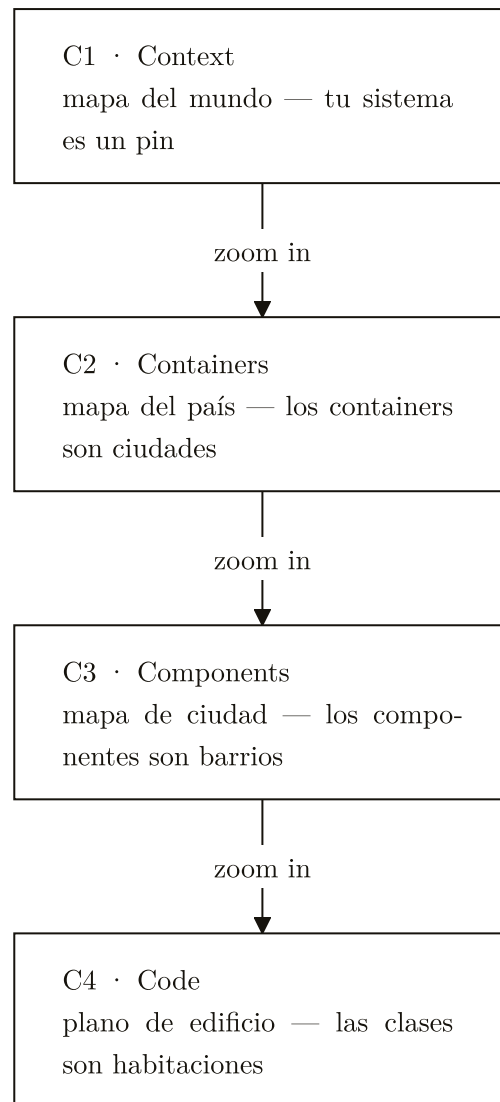


FIGURA 2. *C4 de Brown: cuatro niveles de zoom anidados, de afuera hacia adentro, con la metáfora cartográfica del mundo al plano del edificio.*

EJEMPLO

"Container" ≠ Docker: en C4 un container es una **unidad ejecutable** (proceso/servicio). Puede coincidir con un Docker container, pero también ser una app mobile nativa o un lambda (source: c4-model.md).

CUÁNDO SÍ · VENTAJAS

Ventajas: curva de adopción **baja** (solo 4 tipos de diagrama); **notación libre** con tooling moderno (Structurizr, PlantUML+C4, Mermaid); encaja en wikis ágiles y microservicios. La cátedra prefiere notación **textual** (Structurizr/PlantUML) para que los diagramas vivan en git y evolucionen con el código (source: c4-model.md).

CUÁNDO NO • CUIDADO

Anti-patrones: dibujar los 4 niveles **siempre**, aun en sistemas chicos — regla: solo los niveles que *alguien* va a leer; usar C4 como **reemplazo completo** de la documentación (no cubre decisiones, trade-offs ni QAs); diagramas **desactualizados** — un C2 que no coincide con lo deployado es peor que no tenerlo (source: c4-model.md).

FUENTE

Brown sobre C4 (Code): suele no hacer falta documentarlo — el código es su propia documentación y mantener diagramas de clases sincronizados es costoso. C1-C2-C3 según valor; C4 solo cuando aporta (source: c4-model.md).

§4 4+1 vs C4 + enfoque híbrido

TABLA 3. Dos ejes distintos para el mismo problema

Dimensión	4+1 (Kruchten)	C4 (Brown)
Eje organizador	Tipo de stakeholder	Nivel de zoom
Origen	1995, IEEE Software	~2018, web/blog
Niveles	4 vistas ortogonales + escenarios	4 niveles anidados (C1→C4)
Notación	UML	Libre, tooling moderno
Curva de adopción	Alta (requiere UML)	Baja
Atributos de calidad / trade-offs	Débil — hay que añadirlo	Débil — hay que añadirlo
Encaja mejor en	Proyectos grandes, doc. formal	Wikis ágiles, microservicios

TRADE-OFF

Formalidad ↔ agilidad: 4+1 da rigor y trazabilidad por stakeholder pero a costo de UML y ritual; C4 es liviano y vive en git pero expresa peor las decisiones. Ninguno de los dos cubre explícitamente atributos de calidad ni ADRs (source: c4-model.md).

CUÁNDO SÍ • VENTAJAS

Híbrido recomendado por la cátedra: tomar conceptos de ambos y producir solo las vistas que el contexto exige (source: modelo-4-mas-1.md). En concreto: **C4 C1-C2** como diagramas vivos en el repo + **escenarios clave** (la +1 de 4+1) narrados arriba de los diagramas + **ADRs** para las decisiones significativas + árbol de utilidad para trazar QAs (source: c4-model.md).

§5 ADR — Architecture Decision Record

DEFINICIÓN

ADR: documento corto y **versionado** que registra una decisión arquitectónica con su contexto, alternativas, opción elegida y consecuencias. Popularizado por **Michael Nygard** ("*Documenting Architecture Decisions*", 2011); refinado por Zimmermann y la comunidad MADR (source: adr-architecture-decision-record.md).

Valor central: conservar el porqué. Documenta *por qué* la arquitectura es como es; el C4 documenta *qué es*. Son complementarios (source: adr-architecture-decision-record.md).

Estructura canónica (Nygard 2011)

Title

Número + frase corta (ej. *ADR-007: Adoptar PostgreSQL como motor primario*).

Status

Proposed / Accepted / Deprecated / Superseded by ADR-NNN.

Context

Qué fuerzas están en juego: requisitos, restricciones, consideraciones tecnológicas. La narrativa del problema.

Decision

Qué se decidió. Frase clara, en presente: "*Vamos a hacer X.*"

Consequences

Qué pasa después: ventajas, costos, riesgos, qué se vuelve más fácil y qué más difícil.

Variantes (MADR, Y-statements, Arc42) agregan secciones opcionales: Considered Options, Pros and Cons of the Options, Links a tickets y ADRs relacionados (source: adr-architecture-decision-record.md).

```
# ADR-007: Adoptar PostgreSQL como motor primario

## Status
Accepted

## Context
<Fuerzas en juego: requisitos, restricciones, tecnología.
Necesitamos transacciones ACID y consultas relacionales
complejas; el equipo ya conoce SQL.>

## Decision
Vamos a usar PostgreSQL como motor de persistencia primario.

## Consequences
+ ACID, joins ricos, ecosistema maduro.
- Escalado horizontal más costoso que un NoSQL.
~ Se vuelve más fácil el reporting; más difícil el sharding.

## Links (opcional, MADR)
- Considered Options: PostgreSQL, MongoDB, DynamoDB
- Relacionado: ADR-003 (estilo de replicación)
```

CUÁNDO SÍ • VENTAJAS

Cuándo escribir uno: heurística pragmática — **cuando la decisión es difícil de revertir**. Ejemplos: motor de persistencia o estilo de replicación; estilo arquitectónico (microservicios vs monolito modular); shard key / partitioning; posición CAP (CP vs AP); adopción/abandono de framework, runtime o lenguaje; cambios estructurales en el modelo de vistas 4+1 (source: adr-architecture-decision-record.md).

CUÁNDO NO • CUIDADO

Anti-patrones ADR: documentar todo (incluido qué linter usar) → **burocracia** que mata el hábito; ADRs **sin status** (documento eterno sin marca de obsolescencia); ADR **vago** ("usaremos microservicios cuando tenga sentido" no es una decisión); ADRs en **wiki externa** → perder el versionado junto al código rompe la auditabilidad. Los ADRs viven dentro del repo (source: adr-architecture-decision-record.md).

EN EL PARCIAL

Si piden "documentar una arquitectura": no entregar solo cajas. Combiná **(1)** un diagrama (C4 C1/C2 o una vista 4+1) con **(2)** escenarios que lo crucen y **(3)** al menos un ADR con sus 5 secciones (contexto, decisión, alternativas, consecuencias + status). Justificá el **nivel mínimo**: por qué producís esas vistas y no otras.